# 实验 12　文件系统

## 一、实验目的

（1）掌握 openEuler 文件系统的基本原理

（2）基于 openEuler 系统对文件系统进行分析

## 二、背景知识

### 2.1 文件扩展属性（xattr - Extended attributes）

文件扩展属性（xattr - Extended attributes）提供了一种机制，用来将 key-value 键值对永久地关联到文件；让现有的文件系统得以支持在原始设计中未提供的功能。扩展属性是目前流行的 POSIX 文件系统具有的一项特殊的功能，可以给文件、文件夹添加额外的 Key-value 的键值对，键和值都是字符串并且有一定长度的限制。

xattr 扩展函数，仅仅作用于“支持扩展属性的文件系统”，并在挂载文件系统时，需要开启 xattr。因为扩展属性需要底层文件系统的支持，在使用扩展属性的时候，需要查看文件系统说明文章，看此文件系统是否支持扩展属性，以及对扩展属性命名空间等相关的支持。支持扩展属性常见的文件系统有：ext2、ext3、reiserfs、jfs 和 xfs，这些文件系统对于扩展属性的支持都是可选项。

扩展属性名称的格式是 namespace.attribute，名称空间 namespace 是用来定义不同的扩展属性的类。目前有 security，system，trusted，user 四种扩展属性类：

（1）扩展的安全属性-- security

安全属性名称空间被内核用于安全模块，例如 SELinux。对安全属性的读和写权限依赖于策略的设定，这策略是由安全模块载入的，如果没有载入安全模块，所有的进程都对安全属性有读权限，写权限只有那些有 CAP_SYS_ADMIN（允许执行系统管理任务，如加载或卸载文件系统、设置磁盘配额等）的进程才有。

（2）扩展的系统属性-- system

扩展的系统属性被内核用来存储系统对象，比如说 ACL。对系统属性的读和写权限依赖于策略的设定。

（3）受信任的扩展属性-- trusted

受信任的扩展属性只对那些有 CAP_SYS_ADMIN 的进程可见和可获得。这个类中的属性被用来在用户空间中保存一些普通进程无法得到的信息。

（4）扩展的用户属性-- user

扩展的用户属性被分配给文件和目录用来存储任意的附加信息，比如 mime type、字符集或是文件的编码。用户属性的权限由文件权限位来定义。对于普通文件和目录，文件权限位定义文件内容的访问，对于设备文件来说，它们定义对设备的访问。扩展的用户属性只被用于普通的文件和目录，对用户属性的访问被限定于属主和那些对目录有 sticky 位设置的用户。

**2.2 自定义文件系统**

通常，用户在为自己的系统编译内核时可以把 openEuler 配置为能够识别所有需要的文件系统。但是，文件系统的源代码实际上要么包含在内核映像中，要么作为一个模块被动态装入。VFS 必须对目前已在内核中的所有文件系统类型进行跟踪。这就是通过进行文件系统类型注册来实现的。

**2.3 proc 文件系统**

openEuler 系统上的/proc 目录是一种文件系统，即 proc 文件系统。与其它常见的文件系统不同的是，/proc 是一种伪文件系统（也即虚拟文件系统），存储的是当前内核运行状态的一系列特殊文件，用户可以通过这些文件查看有关系统硬件及当前正在运行进程的信息，甚至可以通过更改其中某些文件来改变内核的运行状态。

基于 /proc 文件系统如上所述的特殊性，其内的文件也常被称作虚拟文件，并具有一些独特的特点。例如，其中有些文件虽然使用查看命令查看时会返回大量信息，但文件本身的大小却会显示为 0 字节。此外，这些特殊文件中大多数文件的时间及日期属性通常为当前系统时间和日期，这跟它们随时会被刷新（存储于 RAM 中）有关。

为了查看及使用上的方便，这些文件通常会按照相关性进行分类存储于不同的目录甚至子目录中，如/proc/scsi 目录中存储的就是当前系统上所有 SCSI 设备的相关信息，/proc/N 中存储的则是系统当前正在运行的进程的相关信息，其中 N 为正在运行的进程 PID（在某进程结束后其相关目录则会消失）。

大多数虚拟文件可以使用文件查看命令如 cat、more 或者 less 进行查看，有些文件信息表述的内容可以一目了然，但也有文件的信息却不怎么具有可读性。不过，这些可读性较差的文件在使用一些命令如 apm、free、lspci 或 top 查看时却可以有着不错的表现。

### 2.4 sysfs 文件系统

内核子系统或设备驱动可以直接编译到内核，也可以编译成模块。如果编译到内核，可以通过内核启动参数来向它们传递参数；如果编译成模块，则可以通过命令行在插入模块时传递参数，或者在运行时，通过 sysfs 来设置或读取模块数据。

sysfs 是一个基于内存的虚拟文件系统，可以看成与 proc、devfs 和 devpty 同类别的文件系统；它的作用是将内核信息以文件的方式提供给用户程序使用。sysfs 文件系统要求总是被挂载在 /sys 挂载点上，这个文件系统不仅可以把设备（devices）和驱动程序（drivers）的信息从内核输出到用户空间，也可以用来对设备和驱动程序做设置。

## 三、实验内容

### 3.1 文件扩展属性模拟

扩展属性给文件、文件夹添加额外的 Key-value 的键值对，键和值定义于 include/uapi/linux/limits.h 文件中：

```
#define XATTR_NAME_MAX   255
#define XATTR_SIZE_MAX    65536
#define XATTR_LIST_MAX    65536
```

即：在保存 xattr 时， key 的长度不能超过 255byte，value 不能超过 64k，总的配对数不能超过 64k。

为了使用扩展属性，需要安装 libattr，然后可以查看当前文件系统类型：

```
[root@localhost ~]# dnf install -y libattr
[root@localhost ~]# df -Th
文件系统                类型      容量  已用  可用 已用% 挂载点
devtmpfs                devtmpfs  1.7G     0  1.7G    0% /dev
tmpfs                   tmpfs     1.7G     0  1.7G    0% /dev/shm
tmpfs                   tmpfs     1.7G  169M  1.5G   10% /run
tmpfs                   tmpfs     1.7G     0  1.7G    0% /sys/fs/cgroup
/dev/mapper/openeuler-root ext4   50G   19G   29G   40% /
tmpfs                   tmpfs     1.7G     0  1.7G    0% /tmp
```

```
/dev/mapper/openeuler-home    ext4          25G    557M     23G       3% /home
/dev/sda1                     ext4          976M   247M    663M      28% /boot
tmpfs                         tmpfs         341M      0    341M       0% /run/user/0
```

用 fdisk -l 查看硬盘及分区信息，在 ext3 和 ext4 文件系统上，可通过命令 tune2fs -l 来检查是否支持扩展属性：

```
[root@localhost ~]# fdisk -l
Disk /dev/sda：80 GiB，85899345920 字节，167772160 个扇区
磁盘型号：VMware Virtual S
单元：扇区 / 1 * 512 = 512 字节
扇区大小(逻辑/物理)：512 字节 / 512 字节
I/O 大小(最小/最佳)：512 字节 / 512 字节
磁盘标签类型：dos
磁盘标识符：0x9d8eebc0
设备          启动       起点          末尾          扇区  大小  Id  类型
/dev/sda1    *          2048     2099199     2097152     1G 83 Linux
/dev/sda2               2099200 167772159 165672960    79G 8e Linux LVM
Disk /dev/mapper/openeuler-root：50.44 GiB，54144270336 字节，105750528 个扇区
单元：扇区 / 1 * 512 = 512 字节
扇区大小(逻辑/物理)：512 字节 / 512 字节
I/O 大小(最小/最佳)：512 字节 / 512 字节
Disk /dev/mapper/openeuler-swap：3.98 GiB，4244635648 字节，8290304 个扇区
单元：扇区 / 1 * 512 = 512 字节
扇区大小(逻辑/物理)：512 字节 / 512 字节
I/O 大小(最小/最佳)：512 字节 / 512 字节
Disk /dev/mapper/openeuler-home：24.63 GiB，26432503808 字节，51625984 个扇区
单元：扇区 / 1 * 512 = 512 字节
扇区大小(逻辑/物理)：512 字节 / 512 字节
I/O 大小(最小/最佳)：512 字节 / 512 字节
[root@localhost ~]# tune2fs -l /dev/mapper/openeuler-root
tune2fs 1.45.6 (20-Mar-2020)
Filesystem volume name:   <none>
Last mounted on:          /
Filesystem UUID:          de7db1c3-32ab-4c31-bf07-d106f3d6f75b
Filesystem magic number:  0xEF53
Filesystem revision #:    1 (dynamic)
Filesystem features:      has_journal ext_attr resize_inode dir_index filetype needs_recovery
extent 64bit flex_bg sparse_super large_file huge_file dir_nlink extra_isize metadata_csum
Filesystem flags:         signed_directory_hash
Default mount options:    user_xattr acl
```

通过检查/dev/mapper/openeuler-root 文件系统参数中的默认挂载选项“Default mount options”是否有对应内容；来确定分区设备的文件系统是否支持扩展属性——上图所示说明，该文件系统支持扩展用户属性 user_xattr、与 ACL

权限。

文件扩展属性的设置如下：

（1）setfattr：设置文件系统对象的扩展属性（无文件系统限制）

语法：setfattr [-h] -n name [-v value] pathname...

setfattr [-h] -x name pathname...

setfattr [-h] --restore=file

- -n name：指定属性名称
- -v value：设置属性值。可以使用三种方法对 value 进行编码：
  - ✓ 如果给定的字符串用双引号引起来，则将其中的字符串视为文本。在这种情况下，其中的反斜线和双引号需转义。任何控制字符都可以编码为“反斜杠后跟三个数字”作为八进制的 ASCII 码。
  - ✓ 如果给定的字符串以 0x 开头，则表示一个十六进制数。
  - ✓ 如果给定的字符串以 0s 开头，则需要 base64 编码。
- -x name：删除属性
- -h：不要遵循符号链接。 如果路径名是符号链接，则不会遵循它，而是将其本身修改为 inode。
- --version：打印 setfattr 的版本并退出。
- --help：打印帮助以解释命令行选项。

（2）getfattr：获取文件系统对象的扩展属性

- -n name：显示指定名称的属性。
- -d：显示所有属性的值。
- -e en：在提取属性之后进行编码，en 的值为 text、hex 和 base64。
  - ✓ en= "text"时，编码为文本的字符串的值用双引号（“）引起来；
  - ✓ en= "hex"时，编码为十六进制的字符串以 0x 为前缀；
  - ✓ en= "base64"时，编码为 base64 的字符串以 0s 为前缀。
- -m pattern：正则表达式匹配的属性显示。默认匹配为'^user\\.'，即匹配 user 名称空间的属性，可以指定为 '-' 或 '.' 匹配所有属性。
- -R：递归显示。

### 3.1.1 主要任务

为 Ext4 文件系统添加扩展属性：

（1）熟悉文件系统扩展属性 EA，查看文件系统是否支持 EA。

（2）使用 setfattr 设置文件系统的用户扩展属性，并设置文本、八进制数、十六进制数与 base64 编码这四种属性值。

（3）使用 getfattr 获取文件系统的用户扩展属性，并在获取属性之后进行 text、hex 和 base64 这三种编码设置。

### 3.1.2 具体步骤

①打开 FinalShell 窗口，创建一个 txt 文件，命名为 file：

```
[root@localhost ~]# touch file.txt
```

②对于不含转义字符 \ 的纯文本属性值，有无双引号限定效果一样:

```
[root@localhost ~]# setfattr -n user.name -v xattr_test file.txt
[root@localhost ~]# setfattr -n user.city -v "Beijing" file.txt
[root@localhost ~]# getfattr -d -m . file.txt
# file: file.txt
user.city="Beijing"
user.name="xattr_test"
```

③对于包含转义字符 \ 的文本属性值，无双引号则不对转义符 \ 进行转义；有双引号则对其进行转义。例如：设置八进制数时，设置属性值为 \012 时，最终识别为文本 012；设置属性值为“\012”时，最终以八进制数的 base64 编码存储。

```
[root@localhost ~]# setfattr -n user.age -v \012 file.txt
[root@localhost ~]# getfattr -d -m . file.txt
# file: file.txt
user.age="012"
user.city="Beijing"
user.name="xattr_test"
```

④设置属性值为十六进制数时，所设置的数的位数必须为偶数，即 0x 或 0X 后的数字必须为偶数位，否则出错。 若设置成功，最终以十六进制数的 base64 编码存储。

```
[root@localhost ~]# setfattr -n user.hex -v 0x123 file.txt
bad input encoding
[root@localhost ~]# setfattr -n user.hex -v 0x0123 file.txt
[root@localhost ~]# getfattr -d -m . file.txt
# file: file.txt
user.age="012"
user.city="Beijing"
user.hex=0sASM=
```

```
user.name="xattr_test"
```

⑤设置属性值为 base64 编码时，所设置的编码必须符合 base64 编码，即 0s 后的编码字符串必须符合 base64 编码，否则出错。 若设置成功，最终以 base64 编码对应的文本信息存储。

```
[root@localhost ~]# setfattr -n user.base64 -v 0sSGVsbG8gd29ybGQh file.txt
[root@localhost ~]# getfattr -d -m . file.txt
# file: file.txt
user.age="012"
user.base64="Hello world!"
user.city="Beijing"
user.hex=0sASM=
user.name="xattr_test"user.city="Beijing"
user.hex=0sASM=
user.name="xattr_test"
```

注：可在 https://base64.us/ 中，将需要设置的属性值进行 base64 编码后，再使用 setfattr 命令设置，注意设置时需在 base64 编码前加 0s 前缀。

⑥使用 getfattr 获取属性后进行 text 编码设置。当未对获取的属性进行编码设置时，直接使用 getfattr -d -m . file.txt 或 getfattr -d file.txt 即可完成。

```
[root@localhost ~]# getfattr -d -m . file.txt
# file: file.txt
user.age="012"
user.base64="Hello world!"
user.city="Beijing"
user.hex=0sASM=
user.name="xattr_test"
[root@localhost ~]# getfattr -d file.txt
# file: file.txt
user.age="012"
user.base64="Hello world!"
user.city="Beijing"
user.hex=0sASM=
user.name="xattr_test"
```

对获取的属性设置 text 编码时，user.age 与 user.hex 的属性值变成了对应的文本值，user.name、user.city、user.base64 都仍保留原来的文本值，结果如下：

```
[root@localhost ~]# getfattr -d -e text file.txt
# file: file.txt
user.age="012"
user.base64="Hello world!"
user.city="Beijing"
user.hex="#"
```

```
user.name="xattr_test"
```

⑦使用 getfattr 获取属性后进行 hex 编码设置。user.age 的属性值由八进制变成了十六进制，user.hex 的属性值还原成了最初设置的原值，user.name、user.city、user.base64 的属性值都转化为对应的十六进制值，结果如下：

```
[root@localhost ~]# getfattr -d -e hex file.txt
# file: file.txt
user.age=0x303132
user.base64=0x48656c6c6f20776f726c6421
user.city=0x4265696a696e67
user.hex=0x0123
user.name=0x78617474725f74657374
```

⑧使用 getfattr 获取属性后进行 base64 编码设置。user.age 和 user.hex 的属性值保留最初的 base64 编码存储，user.name、user.city、user.base64 的属性值都转化为对应的 base64 编码值，结果如下：

```
[root@localhost ~]# getfattr -d -e base64 file.txt
# file: file.txt
user.age=0sMDEy
user.base64=0sSGVsbG8gd29ybGQh
user.city=0sQmVpamluZw==
user.hex=0sASM=
user.name=0seGF0dHJfdGVzdA==
```

### 3.2 自定义文件系统模拟

查看系统中已经注册的文件系统类型：

```
[root@localhost ~]# cat /proc/filesystems
nodev   sysfs
nodev   rootfs
nodev   ramfs
nodev   bdev
nodev   proc
nodev   cpuset
nodev   cgroup
nodev   cgroup2
nodev   tmpfs
nodev   devtmpfs
nodev   configfs
nodev   debugfs
nodev   tracefs
nodev   securityfs
nodev   sockfs
```

```
nodev   dax
nodev   bpf
nodev   pipefs
nodev   hugetlbfs
nodev   devpts
nodev   autofs
nodev   mqueue
nodev   pstore
        ext3
        ext2
        ext4
```

注册函数 register_filesystem 和注销函数 unregister_filesystem 分别用于文件系统类型的注册和注销，都只有一个参数，即代表文件系统类型的一个指向 file_system_type 对象的指针。

（1）头文件：<linux/fs.h>

（2）函数原型：两个函数的原型分别为：

- int register_filesystem(struct file_system_type *)；
- int unregister_filesystem(struct file_system_type * )；

（3）返回值：函数执行成功时，返回 0；否则，返回一个错误值。

（4）参数说明：

每个注册的文件系统都用一个类型为 file_system_type 的对象来表示，其结构体定义如下：

```
struct file_system_type {
    const char *name;文件系统名称
    int fs_flags;
#define FS_REQUIRES_DEV         1
#define FS_BINARY_MOUNTDATA     2
#define FS_HAS_SUBTYPE          4
#define FS_USERNS_MOUNT         8       /* Can be mounted by userns root */
#define FS_USERNS_DEV_MOUNT     16 /* A userns mount does not imply MNT_NODEV */
#define FS_USERNS_VISIBLE       32      /* FS must already be visible */
#define FS_RENAME_DOES_D_MOVE   32768   /* FS will handle d_move() during rename()
internally. */
    struct dentry *(*mount) (struct file_system_type *, int,
                const char *, void *);/*挂载函数指针*/
    void (*kill_sb) (struct super_block *);/*释放超级块函数指针*/
    struct module *owner;
    struct file_system_type * next;
    struct hlist_head fs_supers;
```

```
    struct lock_class_key s_lock_key;
    struct lock_class_key s_umount_key;
    struct lock_class_key s_vfs_rename_key;
    struct lock_class_key s_writers_key[SB_FREEZE_LEVELS];

    struct lock_class_key i_lock_key;
    struct lock_class_key i_mutex_key;
    struct lock_class_key i_mutex_dir_key;
};
```

字段的含义：

| | |
|---|---|
| name | 文件系统类型名 |
| fs_flags | 文件系统类型的标志，可以使用以下标志或其组合： |
| FS_REQUIRES_DEV | 文件系统必须安装在物理磁盘设备上 |
| FS_BINARY_MOUNTDATA | 文件系统使用二进制安装数据 |
| FS_HAS_SUBTYPE | 文件系统含有子类型 |
| FS_USERNS_MOUNT | 可以使用用户命名空间的根进行文件系统的加载 |
| FS_USERNS_DEV_MOUNT | 使用用户空间进行加载不意味着 MNT_NODEV |
| FS_RENAME_DOES_D_MOVE | 重命名操作内部实现使用移动操作 |
| mount | 安装文件系统时，调用的方法 |
| kill_sb | 删除超级块的方法 |
| owner | 指向实现文件系统的模块的指针，一般 THIS_MODULE 填充 |
| next | 指向文件系统类型链表中下一个元素的指针 |
| fs_supers | 具有相同文件系统类型的超级块对象链表的头 |

其它所有 lock_class_key 类型的锁变量，都是对特定的读写操作进行保护，避免同时访问。

ext4fs 文件系统对这个结构体的填充：

```
static struct file_system_type ext4_fs_type = {
    .owner        = THIS_MODULE,
    .name         = "ext4",
    .mount        = ext4_mount,
    .kill_sb      = kill_block_super,
    .fs_flags     = FS_REQUIRES_DEV,
};
```

```
MODULE_ALIAS_FS("ext4");
```

ramfs 文件系统对这个结构体的填充:

```
static struct file_system_type ramfs_fs_type = {
    .owner          = THIS_MODULE,
    .name           = "ramfs",
    .mount          = ramfs_mount,
    .kill_sb        = ramfs_kill_sb,
    .fs_flags       = FS_USERNS_MOUNT,
};
```

一般文件系统的实现，实现这几个字段即可；其它的由内核利用或者填充。

### 3.2.1 主要任务

使用文件系统注册/注销函数，注册一个自定义的文件系统类型。

### 3.2.2 具体步骤

①打开 FinalShell 窗口，创建一个目录，命名为 file_demo1, cd 到该目录下，创建一个 C 文件，命名为 register_newfs，并打开该文件：

```
[root@localhost ~]# mkdir file_demo1
[root@localhost ~]# cd file_demo1
[root@localhost file_demo1]# touch register_newfs.c
[root@localhost file_demo1]# vi register_newfs.c
```

②按“i”键进入 Insert 模式，在 register_newfs.c 文件中写入如下代码，按“Esc”退出 Insert 模式，输入“：wq”保存：

```
#include <linux/module.h>
#include <linux/fs.h>

MODULE_LICENSE("GPL");

static struct file_system_type myfs_type = {
    .name     = "myfs",
    .owner    = THIS_MODULE,
};
MODULE_ALIAS_FS("myfs");

static int __init register_newfs_init(void)
{
    printk("Start register_newfs module...");
    return register_filesystem(&myfs_type);
}
```

```
static void __exit register_newfs_exit(void)
{
    printk("Exit register_newfs module...");
    unregister_filesystem(&myfs_type);
}

module_init(register_newfs_init);
module_exit(register_newfs_exit);
```

③使用命令“vi Makefile”创建并打开 Makefile 文件，按“i”键进入 Insert 模式，在文件中写入如下代码，按“Esc”退出 Insert 模式，输入“：wq”保存：

```
ifneq ($(KERNELRELEASE),)
    obj-m := register_newfs.o
else
    /*选择自己的内核路径*/
    KERNELDIR ?= /root/kernel-4.19.90-2108.8.0
    PWD := $(shell pwd)
default:
    $(MAKE) -C $(KERNELDIR) M=$(PWD) modules
endif
.PHONY:clean
clean:
    -rm *.mod.c *.o *.order *.symvers *.ko
```

④编译并加载 U 盘内核模块：

```
[root@localhost file_demo1]# make
make -C /root/kernel-4.19.90-2108.8.0 M=/root/file_demo1 modules
make[1]: 进入目录“/root/kernel-4.19.90-2108.8.0”
  CC [M]    /root/file_demo1/register_newfs.o
  Building modules, stage 2.
  MODPOST 1 modules
  CC          /root/file_demo1/register_newfs.mod.o
  LD [M]    /root/file_demo1/register_newfs.ko
make[1]: 离开目录“/root/kernel-4.19.90-2108.8.0”
[root@localhost file_demo1]# insmod register_newfs.ko
```

⑤实验结果如下：

```
[root@localhost file_demo1]# cat /proc/filesystems | grep myfs
[root@localhost file_demo1]# insmod register_newfs.ko
[root@localhost file_demo1]# cat /proc/filesystems | grep myfs
nodev    myfs
[root@localhost file_demo1]# rmmod register_newfs.ko
[root@localhost file_demo1]# cat /proc/filesystems | grep myfs
[root@localhost file_demo1]#
```

当未加载内核模块时，当前系统中无自定义的文件系统“myfs”;当加载内

核模块时，当前系统中可打印出自定义的文件系统“myfs”;当卸载内核模块时，当前系统中无自定义的文件系统“myfs”。

⑥卸载模块：

```
[root@localhost file_demo1]# rmmod register_newfs.ko
```

**3.3proc 文件系统模拟**

/proc 目录中的目录操作函数：

（1）proc_mkdir()

功能：用于在/proc 目录中添加一个目录。

头文件：linux/proc_fs.h

函数原型为：

static inline struct proc_dir_entry *proc_mkdir(const char *name, struct proc_dir_entry *parent)

参数说明：

name 建立的目录的名称

parent 父目录指针。如果为 NULL，则在 /proc 目录下建立。

返回值：成功时返回创建的目录的指针；失败时返回 NULL。

（2）proc_dir_entry 结构体

头文件：定义于 fs/proc/internal.h，使用时添加 linux/proc_fs.h 即可。

```
struct proc_dir_entry {
        /*
         * number of callers into module in progress;
         * negative -> it's going away RSN
         */
        atomic_t in_use;
        refcount_t refcnt;
        struct list_head pde_openers;      /* who did ->open, but not ->release */
        /* protects ->pde_openers and all struct pde_opener instances */
        spinlock_t pde_unload_lock;
        struct completion *pde_unload_completion;
        const struct inode_operations *proc_iops;
        const struct file_operations *proc_fops;
        const struct dentry_operations *proc_dops;
        union {
                const struct seq_operations *seq_ops;
                int (*single_show)(struct seq_file *, void *);
        };
```

```
        proc_write_t write;
        void *data;
        unsigned int state_size;
        unsigned int low_ino;
        nlink_t nlink;
        kuid_t uid;
        kgid_t gid;
        loff_t size;
        struct proc_dir_entry *parent;
        struct rb_root subdir;
        struct rb_node subdir_node;
        char *name;
        umode_t mode;
        u8 namelen;
        char inline_name[];
} __randomize_layout;
```

结构体对 proc 文件系统目录项的对象做了定义。分为三个部分：

①proc 目录项的属性，例如

low_ino 此目录项对应的索引节点的值

mode　对应的索引节点的类型和权限模式

size　　对应的索引节点的大小

name　目录项的名称

②proc 目录项的方法，例如

proc_iops　　索引节点的操作的集合

proc_fops　　文件操作的集合

openEuler 内核使用的属性，例如 in_use，pde_unload_lock 等等。

（3）proc_remove()

功能：用于移除 /proc 目录下的一个目录及其子目录

函数原型为：

static inline void proc_remove(struct proc_dir_entry *de)

参数为要删除的目录的指针（即 proc_mkdir()函数的返回值）。

### 3.3.1 主要任务

编写一个模块，在加载模块时，在 /proc 目录下创建一个名称为 myproc 的目录；加载模块后，查看系统中是否在 /proc 目录下成功创建 myproc 目录；加载、卸载模块并查看模块打印信息。

①打开 FinalShell 窗口，创建一个目录，命名为 file_demo2, cd 到该目录下，创建一个 C 文件，命名为 proc_mkdir.c，并打开该文件：

```
[root@localhost ~]# mkdir file_demo2
[root@localhost ~]# cd file_demo2
[root@localhost file_demo2]# touch proc_mkdir.c
[root@localhost file_demo2]# vi proc_mkdir.c
```

②按“i”键进入 Insert 模式，在 proc_mkdir.c 文件中写入如下代码，按“Esc”退出 Insert 模式，输入“：wq”保存：

```
#include <linux/module.h>
#include <linux/proc_fs.h>

MODULE_LICENSE("GPL");

static struct proc_dir_entry *myproc_dir;

static int __init myproc_init(void)
{
	int ret = 0;
	printk("Start proc_mkdir module...");
	myproc_dir = proc_mkdir("myproc",NULL);
	if(myproc_dir == NULL)
		return -ENOMEM;
	return ret;
}

static void __exit myproc_exit(void)
{
	printk("Exit proc_mkdir module...");
	proc_remove(myproc_dir);
}

module_init(myproc_init);
module_exit(myproc_exit);
```

③使用命令“vi Makefile”创建并打开 Makefile 文件，按“i”键进入 Insert 模式，在文件中写入如下代码，按“Esc”退出 Insert 模式，输入“：wq”保存：

```
ifneq ($(KERNELRELEASE),)
	obj-m := proc_mkdir.o
else
	/*选择自己的内核路径*/
	KERNELDIR ?= /root/kernel-4.19.90-2108.8.0
	PWD := $(shell pwd)
```

```
default:
	$(MAKE) -C $(KERNELDIR) M=$(PWD) modules
endif
.PHONY:clean
clean:
	-rm *.mod.c *.o *.order *.symvers *.ko
```

④编译并加载 U 盘内核模块：

```
[root@localhost file_demo2]# make
make -C /root/kernel-4.19.90-2108.8.0 M=/root/file_demo2 modules
make[1]: 进入目录“/root/kernel-4.19.90-2108.8.0”
  CC [M]  /root/file_demo2/proc_mkdir.o
  Building modules, stage 2.
  MODPOST 1 modules
  CC       /root/file_demo2/proc_mkdir.mod.o
  LD [M]  /root/file_demo2/proc_mkdir.ko
make[1]: 离开目录“/root/kernel-4.19.90-2108.8.0”
[root@localhost file_demo2]# insmod proc_mkdir.ko
```

⑤实验结果如下：

```
[root@localhost file_demo2]# find /proc/ -name myproc
[root@localhost file_demo2]# insmod proc_mkdir.ko
[root@localhost file_demo2]# find /proc/ -name myproc
/proc/myproc
[root@localhost file_demo2]# rmmod proc_mkdir.ko
[root@localhost file_demo2]# find /proc/ -name myproc
[root@localhost file_demo2]#
```

当未加载内核模块时，/proc 下无 myproc 目录；当加载内核模块时，/proc 下可查找到 myproc 目录。当卸载内核模块时，/proc 下无 myproc 目录。

⑥卸载模块：

```
[root@localhost file_demo2]# rmmod proc_mkdir.ko
```

### 3.4sysfs 文件系统模拟

sysfs 提供一种机制，使得可以显式地描述内核对象、对象属性及对象间关系。sysfs 有两组接口，一组针对内核，用于将设备映射到文件系统中；另一组针对用户程序，用于读取或操作这些设备。

对于模块而言，声明为 static 的变量都可以通过命令行来设置，但要想在 sysfs 下可见，必须通过宏 module_param 来显式声明。

module_param 宏有三个参数：

- 第一个为参数名，即已经定义的变量名；

第二个参数则为变量类型，可用的类型有 byte, short, ushort, int, uint, long, ulong, charp 和 bool 或 invbool，分别对应于 c 语言类型 char, short, unsigned short, int, unsigned int, long, unsigned long, char *

- 和 int，用户也可以自定义类型 XXX（如果用户自己定义了 param_get_XXX，param_set_XXX 和 param_check_XXX）。
- 第三个参数用于指定访问权限，如果为 0，该参数将不出现在 sysfs 文件系统中，允许的访问权限为 S_IRUSR， S_IWUSR，S_IRGRP，S_IWGRP，S_IROTH 和 S_IWOTH 的组合，它们分别对应于用户读，用户写，用户组读，用户组写，其他用户读和其他用户写，因此用文件的访问权限设置是一致的。

module_param 宏使用示例：

```
module_param(a, int, 0);
MODULE_PARM_DESC(a, "An invisible int under sysfs");
module_param(b, int, S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH);
MODULE_PARM_DESC(b, "An visible int under sysfs");
module_param(c, charp, S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH);
MODULE_PARM_DESC(c, "An visible string under sysfs");
```

注：MODULE_PARM_DESC()是对模块的参数进行描述，使用 modinfo 查看模块信息时可见。

当模块加载后，可通过/sys 目录与模块进行交互，通常的做法是使用 echo 向内核传递参数。例如：

```
echo -n 'example' > /sys/module/module_exam/parameters/c
```

说明：

- 用 echo 传递字符串时，使用-n 选项；
- 指定传递参数时，在/sys/module/目录下找到自定义的模块名（如此处示例的 module_exam），模块名目录下的 parameters 目录，即为该模块的参数列表，选定需要传递的参数名即可（如此处示例的参数 c）；

必须使用 su 切换到 root 模式下，使用 sudo 命令会出现权限受限的问题。

### 3.4.1 主要任务

使用 sysfs 文件系统传递内核模块参数。编写一个模块，该模块有三个参数：一个为字符串型，两个为整型。两个整型中，一个在/sys 下不可见。加载模块后，使用 echo 向模块传递参数值来改变指定参数的值。加载、卸载模块并查看模块

打印信息。

### 3.4.2 具体步骤

①打开 FinalShell 窗口，创建一个目录，命名为 file_demo3, cd 到该目录下，创建一个 C 文件，命名为 sysfs_exam.c，并打开该文件：

```
[root@localhost ~]# mkdir file_demo3
[root@localhost ~]# cd file_demo3
[root@localhost file_demo3]# touch sysfs_exam.c
[root@localhost file_demo3]# vi sysfs_exam.c
```

②按“i”键进入 Insert 模式，在 sysfs_exam.c 文件中写入如下代码，按“Esc”退出 Insert 模式，输入“：wq”保存：

```
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/stat.h>

MODULE_LICENSE("GPL");
static int a = 0;
static int b = 0;
static char * c = "Hello, World";
module_param(a, int, 0);
MODULE_PARM_DESC(a, "An invisible int under sysfs");
module_param(b, int, S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH);
MODULE_PARM_DESC(b, "An visible int under sysfs");
module_param(c, charp, S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH);
MODULE_PARM_DESC(c, "An visible string under sysfs");
static int __init sysfs_exam_init(void)
{
	printk("Start sysfs_exam module...");
	printk("a = %d\n", a);
	printk("b = %d\n", b);
	printk("c = '%s'\n", c);
	return 0;
}
static void __exit sysfs_exam_exit(void)
{
	printk("Exit sysfs_exam module...");
	printk("a = %d\n", a);
	printk("b = %d\n", b);
	printk("c = '%s'\n", c);
}

module_init(sysfs_exam_init);
```

```
module_exit(sysfs_exam_exit);
```

③使用命令“vi Makefile”创建并打开 Makefile 文件，按“i”键进入 Insert 模式，在文件中写入如下代码，按“Esc”退出 Insert 模式，输入“：wq”保存：

```
ifneq ($(KERNELRELEASE),)
    obj-m := sysfs_exam.o
else
    KERNELDIR ?= /root/kernel-4.19.90-2108.8.0
    PWD := $(shell pwd)
default:
    $(MAKE) -C $(KERNELDIR) M=$(PWD) modules
endif
.PHONY:clean
clean:
    -rm *.mod.c *.o *.order *.symvers *.ko
```

④编译并加载 U 盘内核模块：

```
[root@localhost file_demo3]# make
make -C /root/kernel-4.19.90-2108.8.0 M=/root/file_demo3 modules
make[1]: 进入目录“/root/kernel-4.19.90-2108.8.0”
  CC [M]    /root/file_demo3/sysfs_exam.o
  Building modules, stage 2.
  MODPOST 1 modules
  CC          /root/file_demo3/sysfs_exam.mod.o
  LD [M]    /root/file_demo3/sysfs_exam.ko
make[1]: 离开目录“/root/kernel-4.19.90-2108.8.0”
[root@localhost file_demo3]# insmod sysfs_exam.ko
```

⑤实验结果如下：

```
[root@localhost file_demo3]# find /sys/module/ -name 'sysfs_exam'
[root@localhost file_demo3]# insmod sysfs_exam.ko
[root@localhost file_demo3]# find /sys/module/ -name 'sysfs_exam'
/sys/module/sysfs_exam
[root@localhost file_demo3]# modinfo sysfs_exam.ko
filename:           /root/file_demo3/sysfs_exam.ko
license:            GPL
srcversion:         3088958EF3BE7DAB164D005
depends:
retpoline:          Y
name:               sysfs_exam
vermagic:           4.19.90 SMP mod_unload modversions
parm:               a:An invisible int under sysfs (int)
parm:               b:An visible int under sysfs (int)
parm:               c:An visible string under sysfs (charp)
[root@localhost file_demo3]# ls -al /sys/module/sysfs_exam/parameters/
```

```
总用量 0
drwxr-xr-x 2 root root       0   8 月  22 21:31 .
drwxr-xr-x 6 root root       0   8 月  22 21:26 ..
-rw-r--r-- 1 root root 4096   8 月  22 21:31 b
-rw-r--r-- 1 root root 4096   8 月  22 21:31 c
[root@localhost file_demo3]# cat /sys/module/sysfs_exam/parameters/c
Hello, World
[root@localhost file_demo3]# echo -n 'Happy new year!'>
/sys/module/sysfs_exam/parameters/c
[root@localhost file_demo3]# cat /sys/module/sysfs_exam/parameters/c
Happy new year!
```

当未加载内核模块时，当前系统中无自定义的内核模块“sysfs_exam”；当加载内核模块时，当前系统可见自定义的文件系统“sysfs_exam”。

使用 modinfo 查看内核模块信息，可见 a、b、c 三个参数的描述：

```
parm:           a:An invisible int under sysfs (int)
parm:           b:An visible int under sysfs (int)
parm:           c:An visible string under sysfs (charp)
```

即参数 a 在/sys 中不可见，b、c 在/sys 中可见，输入以下命令：

```
ls -al /sys/module/sysfs_exam/parameters/
```

输出列表可验证上述现象。

可使用 echo 向模块传递参数值来改变指定参数的值。

⑥卸载模块：

```
[root@localhost file_demo3]# rmmod sysfs_exam.ko
```

# 四、实验讨论

1、在什么情况下需要使用文件扩展属性？
2、自定义文件系统有哪些优点和应用场景？